Data Mining

Week 11

Submitter - Himanshu Singh

Fraud Waste Abuse Detection in Healthcare Insurance - Project Milestone Final

Model Building and Evaluation

Milestone 1 **********

Problem Statement

The primary objective of this project is to develop a predictive model to identify Fraud, Waste, and Abuse (FWA) among healthcare providers. By analyzing claims data, the project aims to "Predict potentially fraudulent providers" while uncovering the most significant variables that signal suspicious behavior. This intelligence allows organizations to flag high-risk claims for in-depth investigation, ensuring that resources are directed toward the most likely instances of financial leakage.

Source of data: Elsevier''s Mendeley Data repository is a participating member of the National Institutes of Health (NIH) Office of Data Science Strategy (ODSS) GREI project. liang, tim (2024), “HEALTHCARE PROVIDER FRAUD DETECTION ANALYSIS”, Mendeley Data, V1, doi: 10.17632/gsn2hyty37.1 https://data.mendeley.com/datasets/gsn2hyty37/1

In [323…

```
import pandas as pd
import os
import numpy as np
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
from tqdm import tqdm
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn import metrics
from sklearn.metrics import roc_curve, auc , f1_score
from sklearn.metrics import confusion_matrix
import joblib
```

In [324…

```
data = pd.read_csv('Train.csv')
data_beneficiary = pd.read_csv('Train_Beneficiarydata.csv')
data_inpatient = pd.read_csv('Train_Inpatientdata.csv')
data_outpatient = pd.read_csv('Train_Outpatientdata.csv')
```

Exploratory Data Analysis

We will try to perform EDA (Exploratory Data Analysis) on the dataset we have

- Diagnosis Codes (primarily ICD-10 codes) are the standardized alphanumeric strings used to describe a patient's medical condition, symptoms, or injuries.
- Procedure Codes are standardized numeric or alphanumeric codes used to identify the specific medical, surgical, or diagnostic services provided to a patient. While Diagnosis Codes explain "why" the patient is there, Procedure Codes explain "what" was done to them.
- Reimbursement of the Claim Amount.
- Labelled data of Potential Fraud Providers.
- Fraud case with respect to Labelled Fraud Provider data.
- Money Lost in due to Fraud Cases.

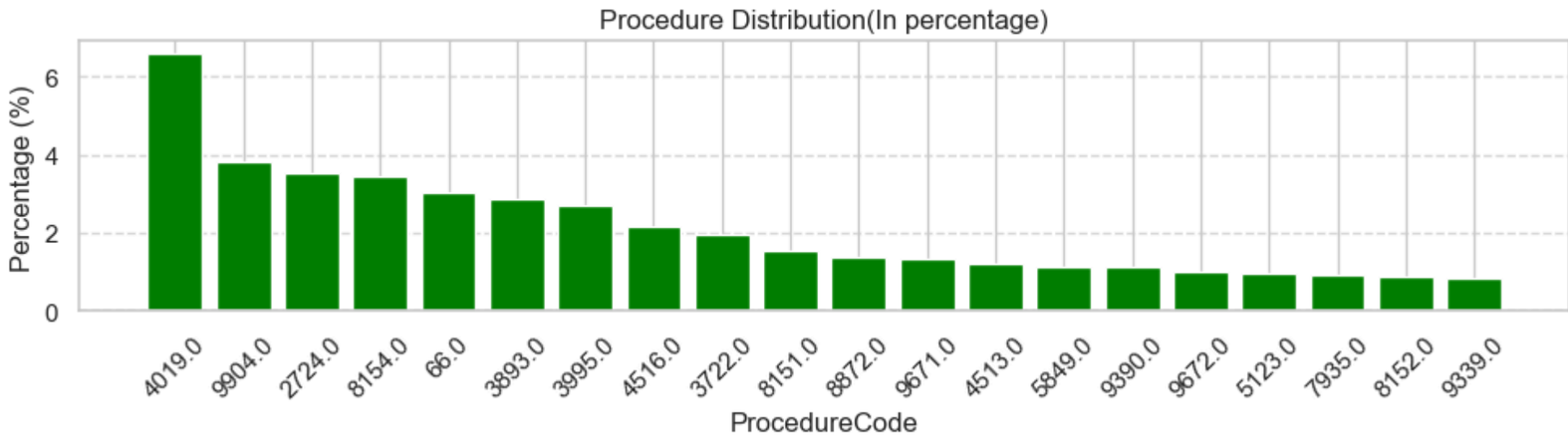
In [325…

```
def get_year(date):
    """This function returns year from date"""
    date = str(date)
    return(int(date[:4]))

def get_month(date):
    """This function returns month from date"""
    date = str(date)
    return (int(date[5:7]))
# Function to analyse categorical columns

def analyse_cat_columns(dataset, col_to_analyse, prefix='', title='Distribution in percentage',
                        top_val=30, y_lim=None, color='green'):
    """Plots a bar graph for the percentage distribution of a categorical column."""

    # Calculate counts and convert to percentage in one step
    counts = dataset[col_to_analyse].value_counts().head(top_val)
    percentages = (counts / len(dataset)) * 100

    # Create the plot labels with the optional prefix
    labels = [f"{prefix}{label}" for label in percentages.index]

    # Plotting
    plt.figure(figsize=(10, 3))
    plt.bar(labels, percentages.values, color=color)

    # Formatting
    plt.title(title)
    plt.ylabel('Percentage (%)')
    plt.xlabel(col_to_analyse)
    plt.xticks(rotation=45)
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    if y_lim is not None:
        plt.yticks(y_lim)

    plt.tight_layout()
    plt.show()
```

In [326…

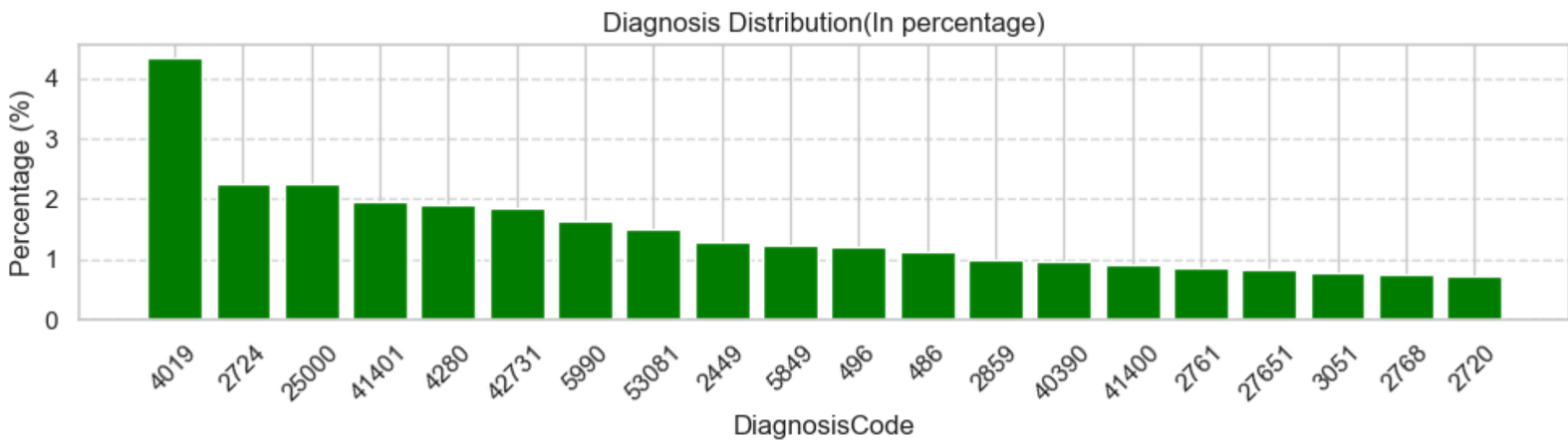
```
# Select all columns that start with 'ClmProcedureCode'
proc_cols = [col for col in data_inpatient.columns if 'ClmProcedureCode' in col]

# Concatenate them all and drop NaNs
```

```
inpatient_df = pd.DataFrame(
    pd.concat([data_inpatient[col] for col in proc_cols], axis=0).dropna(),
    columns=['ProcedureCode']
)
analyse_cat_columns(inpatient_df,col_to_analyse='ProcedureCode' ,title = 'Procedure Distribution(In percentage)',top_val = 20)
```



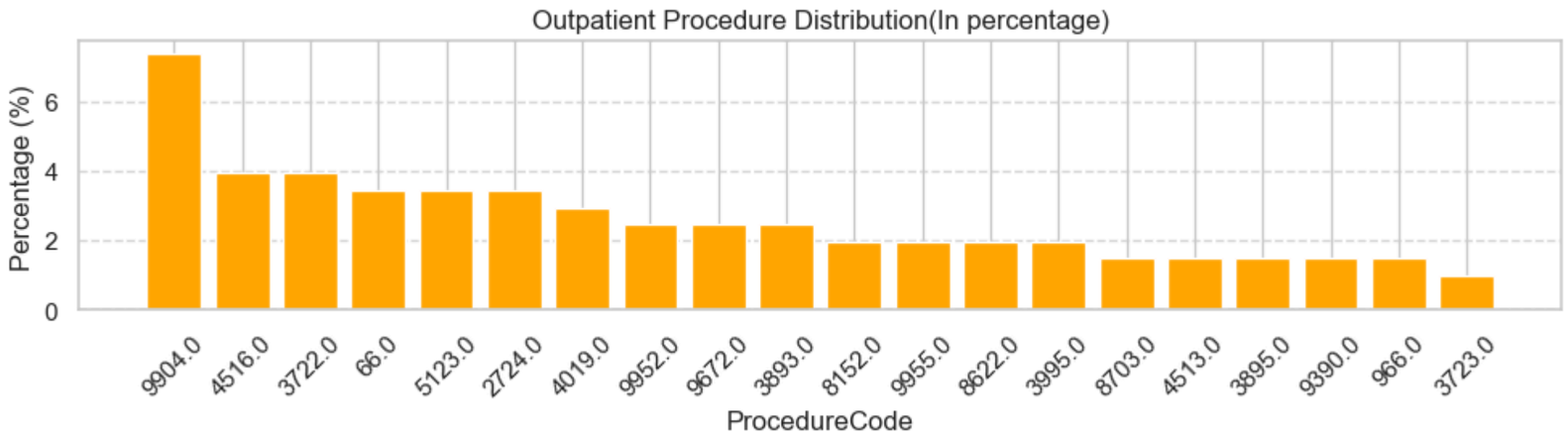
In [327…

```
# Automatically grabs ClmDiagnosisCode_1 through ClmDiagnosisCode_10
diag_cols = data_inpatient.filter(like='ClmDiagnosisCode').columns

# Concatenate into a single Series and remove missing values
inpatient_diag_df = pd.DataFrame(
    pd.concat([data_inpatient[col] for col in diag_cols], axis=0).dropna(),
    columns=['DiagnosisCode']
)
analyse_cat_columns(inpatient_diag_df,col_to_analyse='DiagnosisCode' , title = 'Diagnosis Distribution(In percentage)',top_val = 20, color ='green')
```



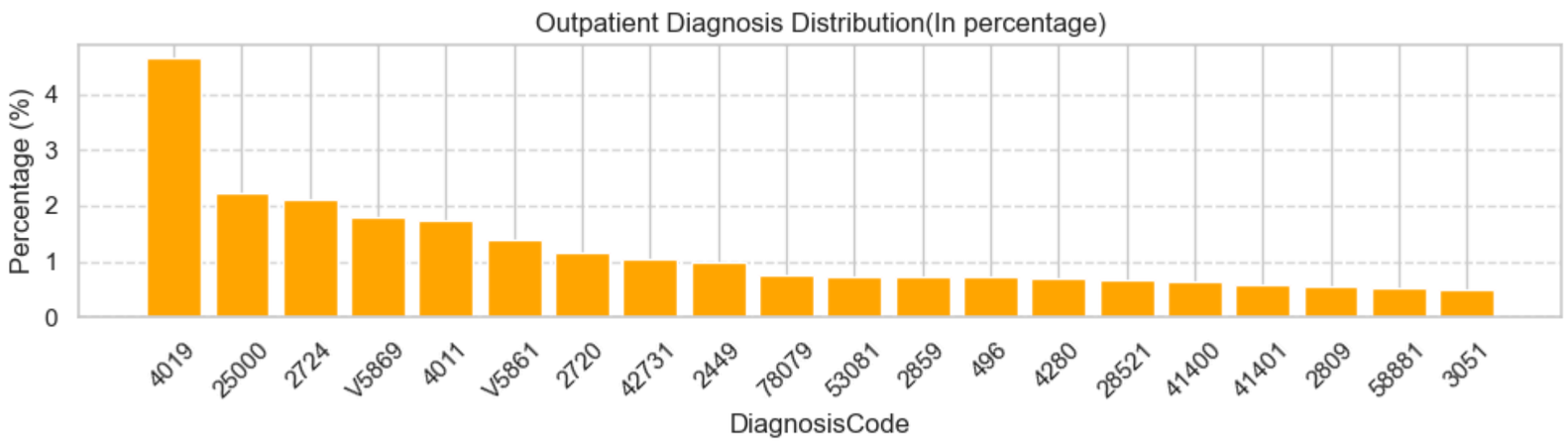
In [328…

```
# Select all columns that start with 'ClmProcedureCode'
proc_cols = [col for col in data_outpatient.columns if 'ClmProcedureCode' in col]

# Concatenate them all and drop NaNs
outpatient_df = pd.DataFrame(
    pd.concat([data_outpatient[col] for col in proc_cols], axis=0).dropna(),
    columns=['ProcedureCode']
)
analyse_cat_columns(outpatient_df,col_to_analyse='ProcedureCode' ,title = 'Outpatient Procedure Distribution(In percentage)',top_val = 20, color='orange')
```



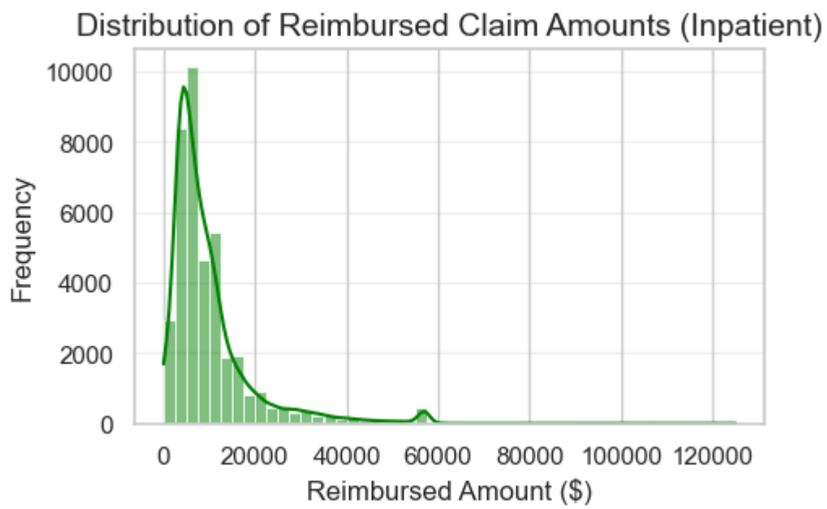
In [329…

```
# Automatically grabs ClmDiagnosisCode_1 through ClmDiagnosisCode_10
diag_cols = data_outpatient.filter(like='ClmDiagnosisCode').columns

# Concatenate into a single Series and remove missing values
outpatient_diag_df = pd.DataFrame(
    pd.concat([data_outpatient[col] for col in diag_cols], axis=0).dropna(),
    columns=['DiagnosisCode']
)
analyse_cat_columns(outpatient_diag_df,col_to_analyse='DiagnosisCode' , title = 'Outpatient Diagnosis Distribution(In percentage)',top_val = 20, color='orange')
```



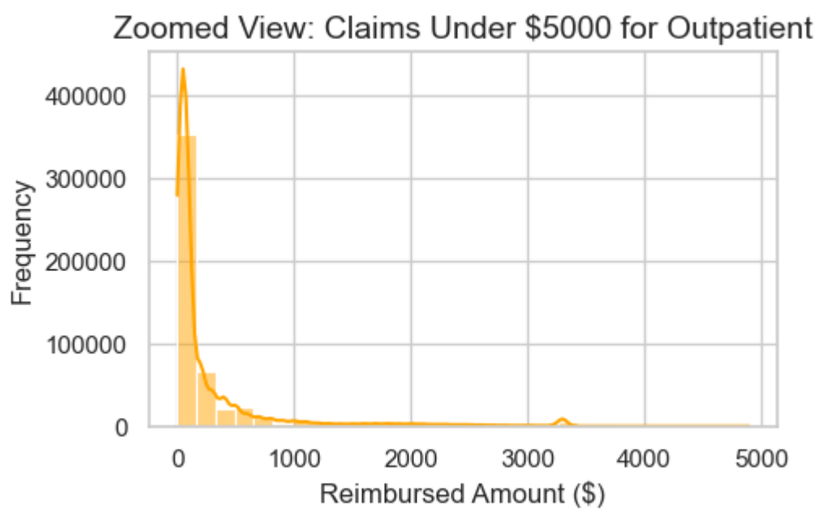
In [330…

```
# Checking distribution of Claim amount for Inpatient Claims
plt.figure(figsize=(5, 3))
# Using histplot for a more modern, detailed view
sns.histplot(data_inpatient['InscClaimAmtReimbursed'], kde=True, color='green', bins=50)

plt.title('Distribution of Reimbursed Claim Amounts (Inpatient)', fontsize=14)
plt.xlabel('Reimbursed Amount ($)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(axis='y', alpha=0.3)
plt.show()
```



In [331…

```
# Checking distribution of Claim amount for Outpatient Claims
# Create a temporary view of only small claims
small_claims = data_outpatient[data_outpatient['InscClaimAmtReimbursed'] < 5000]

plt.figure(figsize=(5, 3))
sns.histplot(small_claims['InscClaimAmtReimbursed'], kde=True, color='orange', bins=30)

plt.title('Zoomed View: Claims Under $5000 for Outpatient', fontsize=14)
plt.xlabel('Reimbursed Amount ($)')
plt.ylabel('Frequency')
plt.show()
```



In [332…

```
#check all the columns of the labeled  dataset and top 5 data points
print('Columns in this dataset are : ')
print(data.columns)
```

```
Columns in this dataset are :
Index(['Provider', 'PotentialFraud'], dtype='object')
```

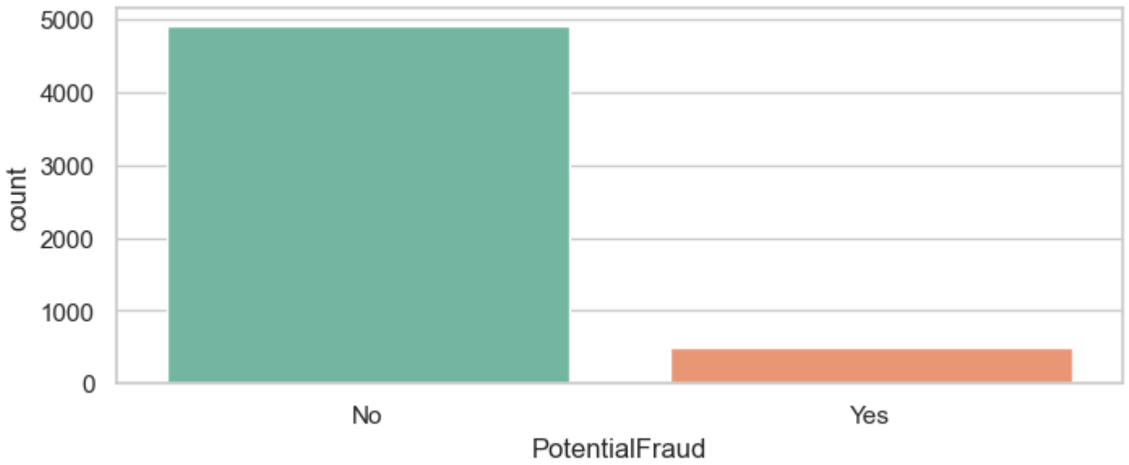
In [333…

```
# Check the distribution of the target variable
plt.figure(figsize=(8, 3))
```

```
sns.countplot(x='PotentialFraud',data=data, palette='Set2')
```

Out[333…

```
<Axes: xlabel='PotentialFraud', ylabel='count'>
```



Percentage of fraudulent encounters Inpatient / Outpatient

In [334…

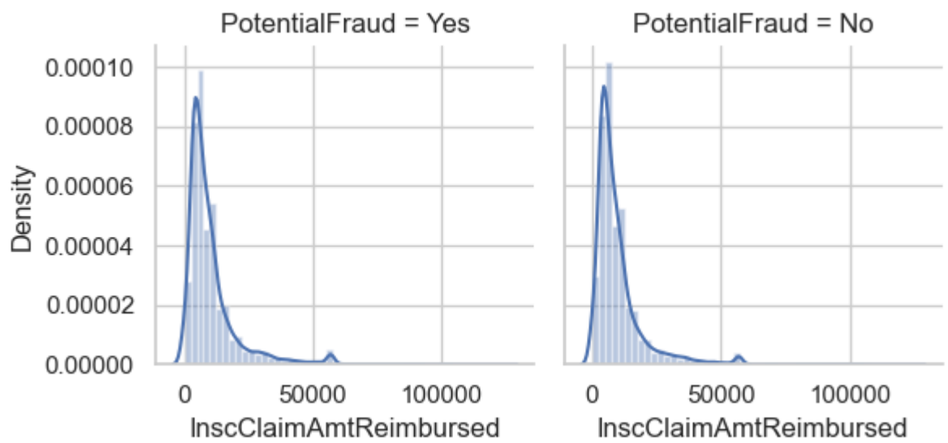
```
# Now we will filter out only the fraudulent providers to analyze their characteristics in more detail.
only_fraud_provider = data.loc[data['PotentialFraud']=='Yes']
fraud_provider_inpatient_df = pd.merge(data_inpatient, only_fraud_provider, how='inner', on='Provider')
print("Percentage of fraud cases related to inpatient data : ",(fraud_provider_inpatient_df.shape[0]/data_inpatient.shape[0])*100)

fraud_provider_outpatient_df = pd.merge(data_outpatient, only_fraud_provider, how='inner', on='Provider')
print("Percentage of fraud cases related to outpatient data : ",(fraud_provider_outpatient_df.shape[0]/data_outpatient.shape[0])*100)
```

```
Percentage of fraud cases related to inpatient data :  57.81983495577408
Percentage of fraud cases related to outpatient data :  36.58112130290089
```

Calculating Money lost in Fraud for both Inpatient and outpatient data

In [335…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.
inpat_labeled_provider_df = pd.merge(data_inpatient , data , how='inner', on='Provider')
sns.FacetGrid(inpat_labeled_provider_df, col='PotentialFraud',height=3).map(sns.distplot, "InscClaimAmtReimbursed",).add_legend()
plt.show()
plt.tight_layout()
Total_money_lost = inpat_labeled_provider_df.loc[inpat_labeled_provider_df['PotentialFraud']== 'Yes'].InscClaimAmtReimbursed.sum()
print("Total money lost : ",Total_money_lost)
```




```
Total money lost :  241288510
<Figure size 640x480 with 0 Axes>
```

In [336…

```
# calculating total money lost outpatient fraudulent encounters
outpat_labeled_provider_df = pd.merge(data_outpatient , data , how='inner', on='Provider')
Total_money_lost = outpat_labeled_provider_df.loc[outpat_labeled_provider_df['PotentialFraud']== 'Yes'].InscClaimAmtReimbursed.sum()
print("Total money lost : ",Total_money_lost)
```

```
Total money lost :  54392610
```

In [337…

```
print('Total money lost as per the data = ','241,288,510 + 54,392,610, That is around 295 Million' )
```

```
Total money lost as per the data =  241,288,510 + 54,392,610, That is around 295 Million
```

Milestone 2 **********

Data preprocessing and Feature Engineering

In [338…

```
# Create a binary target variable for whether the claim was admitted (inpatient) or not (outpatient)
data_inpatient['is_admitted'] = 1
data_outpatient['is_admitted'] = 0

# Merge in_pt, out_pt and ben df into a single patient dataset
data1 = pd.merge(data_inpatient, data_outpatient,
                    left_on = [ idx for idx in data_outpatient.columns if idx in data_inpatient.columns],
                    right_on = [ idx for idx in data_outpatient.columns if idx in data_inpatient.columns],
                    how = 'outer').\
          merge(data_beneficiary,left_on='BeneID',right_on='BeneID',how='inner')

# Replace values with a binary annotation
data1 = data1.replace({'ChronicCond_Alzheimer': 2, 'ChronicCond_Heartfailure': 2, 'ChronicCond_KidneyDisease': 2,
                   'ChronicCond_Cancer': 2, 'ChronicCond_ObstrPulmonary': 2, 'ChronicCond_Depression': 2,
                   'ChronicCond_Diabetes': 2, 'ChronicCond_IschemicHeart': 2, 'ChronicCond_Osteoporasis': 2,
                   'ChronicCond_rheumatoidarthritis': 2, 'ChronicCond_stroke': 2, 'Gender': 2 },
                  0)
# Replace values with a binary annotation
data1 = data1.replace({'RenalDiseaseIndicator': 'Y'}, 1).astype({'RenalDiseaseIndicator': 'int64'})#array([0, 1], dtype=int64)
#merging the dataset created in above step with data
data_final = pd.merge(data1, data , how = 'outer', on = 'Provider' )
# adding extra column target, having binary annotation
```

```
data_final['target'] = data_final['PotentialFraud']

# representing potential fraud and 0 representing non potential fraud
data_final['target'] = data_final['target'].map({"Yes":1,"No":0})
data_final['target'] = data_final['target'].astype('category',copy=False)
```

In [339…

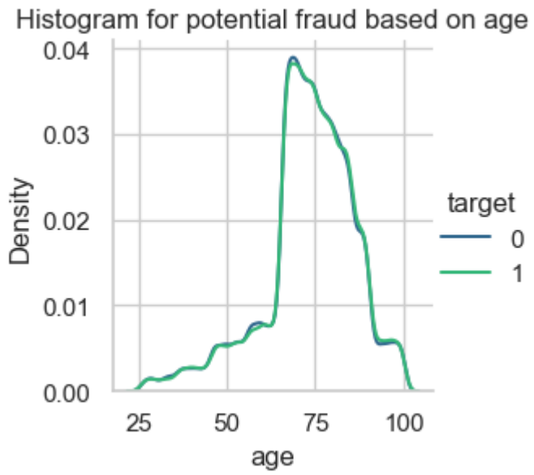
```
# checking the distribution of target variable in the final dataset
print(data_final.PotentialFraud.value_counts())
plt.figure(figsize=(8, 3))
sns.countplot(x='PotentialFraud', data=data_final, palette='Set2', hue='target')
```

```
PotentialFraud
No     345415
Yes    212796
Name: count, dtype: int64
```

Out[339… <Axes: xlabel='PotentialFraud', ylabel='count'>

In [340…

```
# #Adding a column is_dead  = 1  DOD is given else is_dead = 0
# data_final.loc[data_final['DOD'].isnull(), 'Is_Dead'] = '0'
# data_final.loc[(data_final['DOD'].notnull()), 'Is_Dead'] = '1'

data_final['DOB'] =  pd.to_datetime(data_final['DOB'], format='%Y-%m-%d')
data_final['ClaimStartDt'] = pd.to_datetime(data_final['ClaimStartDt'], format='%Y-%m-%d')

#now = pd.to_datetime('2020-09-01' , format = '%Y-%m-%d')
data_final['DOB'] = data_final['DOB'].where(data_final['DOB'] < data_final['ClaimStartDt'])
data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']).dt.days / 365.25
#data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']) / np.timedelta64(1, 'Y')
#data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']).astype('<m8[Y]')

#plotting age
sns.FacetGrid(data_final, hue="target" , height=3, palette='viridis').map(sns.kdeplot, "age",).add_legend()
plt.title('Histogram for potential fraud based on age')
plt.show()
plt.tight_layout()
```



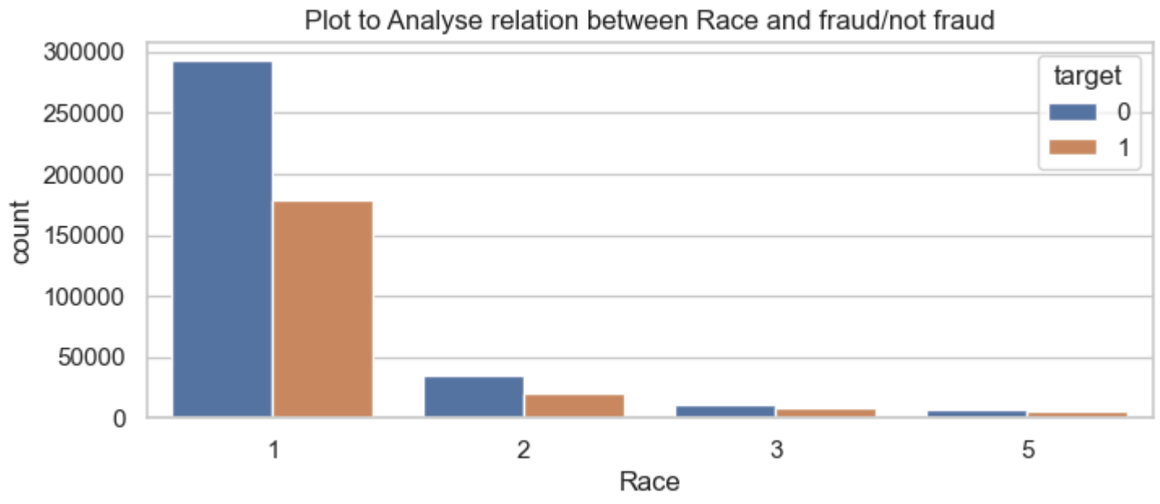

```
<Figure size 640x480 with 0 Axes>
```

In [341…

```
# Now we will analyze the relationship between Race and potential fraud cases in the dataset.
plt.figure(figsize=(8, 3))
ax = sns.countplot(x='Race',data=data_final,hue='target')
ax.set_title("Plot to Analyse relation between Race and fraud/not fraud")
```

Out[341…

```
Text(0.5, 1.0, 'Plot to Analyse relation between Race and fraud/not fraud')
```



In [342…

```
# # Since we didn't have any use for DOD and DOB columns in our analysis, we will drop these columns from the final dataset.
data_final = data_final.drop(['DOD'], axis = 1)
data_final = data_final.drop(['DOB'], axis = 1)
```

In [343…

```
#calculating number of days a patient was admitted in the hospital and then removing admit and discharge date
#outpatients dont get admitted so we keep 0 for them
data_final['AdmissionDt'] = pd.to_datetime(data_final['AdmissionDt'] , format = '%Y-%m-%d')
data_final['DischargeDt'] = pd.to_datetime(data_final['DischargeDt'],format = '%Y-%m-%d')
data_final['Num_admit_days'] = ((data_final['DischargeDt'] - data_final['AdmissionDt']).dt.days)+1
data_final.loc[data_final['is_admitted'] == 0, 'Num_admit_days'] = '0'

data_final = data_final.drop(['DischargeDt'], axis = 1)
data_final = data_final.drop(['AdmissionDt'], axis = 1)
```

In [344…

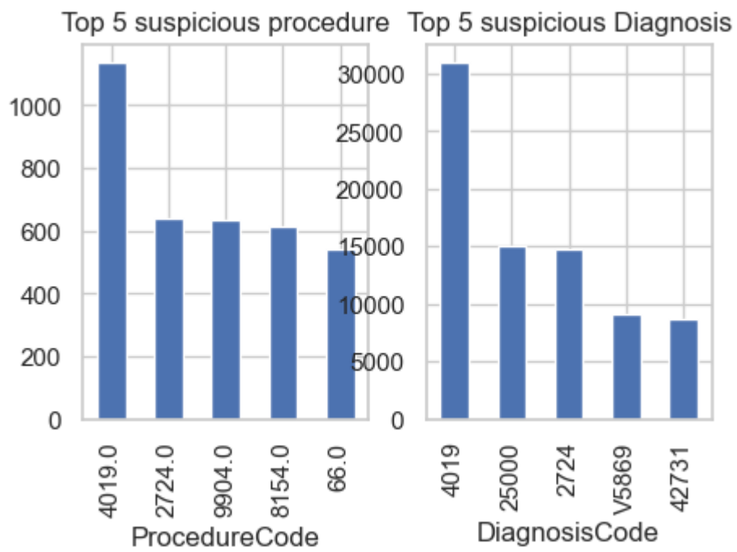
```
#replacing NaN value with 0
data_final.loc[data_final['DeductibleAmtPaid'].isnull(),'DeductibleAmtPaid'] = 0
```

In [345…

```
# Now we will analyze the top procedure codes and diagnosis codes associated with potential fraud cases in the dataset.
potential_fraud_df =data_final.loc[data_final['PotentialFraud'] == 'Yes']

df1 = pd.DataFrame(columns = ['ProcedureCode'])
df1['ProcedureCode'] = pd.concat([potential_fraud_df["ClmProcedureCode_1"],
                                  potential_fraud_df["ClmProcedureCode_2"],
                                  potential_fraud_df["ClmProcedureCode_3"],
                                  potential_fraud_df["ClmProcedureCode_4"],
                                  potential_fraud_df["ClmProcedureCode_5"],
                                  potential_fraud_df["ClmProcedureCode_6"]], axis=0, sort=True).dropna()

df2 = pd.DataFrame(columns = ['DiagnosisCode'])
df2['DiagnosisCode'] = pd.concat([potential_fraud_df["ClmDiagnosisCode_1"],
                                  potential_fraud_df["ClmDiagnosisCode_2"],
                                  potential_fraud_df["ClmDiagnosisCode_3"],
                                  potential_fraud_df["ClmDiagnosisCode_4"],
                                  potential_fraud_df["ClmDiagnosisCode_5"],
                                  potential_fraud_df["ClmDiagnosisCode_6"],
                                  potential_fraud_df["ClmDiagnosisCode_7"],
                                  potential_fraud_df["ClmDiagnosisCode_8"],
                                  potential_fraud_df["ClmDiagnosisCode_9"],
                                  potential_fraud_df["ClmDiagnosisCode_10"]], axis=0, sort=True).dropna()

plt.figure(figsize=(5, 3))
plt.subplot(121)
df1.ProcedureCode.value_counts().head(5).plot(kind = 'bar' , title ='Top 5 suspicious procedure')

plt.subplot(122)
df2.DiagnosisCode.value_counts().head(5).plot(kind = 'bar' , title ='Top 5 suspicious Diagnosis')
```

Out[345…

```
<Axes: title={'center': 'Top 5 suspicious Diagnosis'}, xlabel='DiagnosisCode'>
```



In [346…

```
# Defining a list of all diagnosis and procedure code columns to replace NaN values with 0

Diag_proce_col = ['ClmAdmitDiagnosisCode', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_10',
```

```
                  'ClmDiagnosisCode_2', 'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4',
                  'ClmDiagnosisCode_5', 'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7',
                  'ClmDiagnosisCode_8', 'ClmDiagnosisCode_9', 'ClmProcedureCode_1',
                  'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                  'ClmProcedureCode_5', 'ClmProcedureCode_6']

data_final[Diag_proce_col]= data_final[Diag_proce_col].replace({np.nan:0})
```

In [347…

```
# Now we will create new features based on the top 5 procedure codes and diagnosis codes associated with potential fraud cases in the dataset.
def encoded_cat(dataset, feature_to_encode='',col_list=[]):
    """This function returns top 5 cat column useful in determining potential fraud"""
    outer_list =[]
    for col in col_list:
        list_1 = list()

        for item in list(dataset[col]):
            if str(item) == str(feature_to_encode):
                list_1.append(1)
            else:
                list_1.append(0)

        outer_list.append(list_1)

    li_sum = np.array([0]*558211)

    for i in range(0,len(outer_list)):
        li1 = np.array(outer_list[i])
        li_sum = li_sum + li1

    return li_sum

procedure_col = ['ClmProcedureCode_1','ClmProcedureCode_2','ClmProcedureCode_3','ClmProcedureCode_4','ClmProcedureCode_5','ClmProcedureCode_6']
diagnosis_col = ['ClmDiagnosisCode_1','ClmDiagnosisCode_2','ClmDiagnosisCode_3','ClmDiagnosisCode_4','ClmDiagnosisCode_5','ClmDiagnosisCode_6','ClmDiagnosisCode_7','ClmDiagnos

#top 5 procedure
# 4019.0, 2724.0, 9904.0, 8154.0, 66.0
data_final['pr_4019'] = encoded_cat(data_final,'4019.0',procedure_col)
data_final['pr_2724'] = encoded_cat(data_final,'2724.0',procedure_col)
data_final['pr_9904'] = encoded_cat(data_final,'9904.0',procedure_col)
data_final['pr_8154'] = encoded_cat(data_final,'8154.0',procedure_col)
data_final['pr_66'] =   encoded_cat(data_final,'66.0',procedure_col)

#top 5 diagnosis
#4019,25000,2724,V5869,42731
data_final['di_4019'] = encoded_cat(data_final,'4019',diagnosis_col)
data_final['di_25000'] = encoded_cat(data_final,'25000',diagnosis_col)
data_final['di_2724'] = encoded_cat(data_final,'2724',diagnosis_col)
data_final['di_V5869'] = encoded_cat(data_final,'V5869',diagnosis_col)
data_final['di_42731'] =   encoded_cat(data_final,'42731',diagnosis_col)
```

In [348…

```
# Now we will replace all the values in the diagnosis and procedure code columns with binary annotations, where 1 indicates the presence of a code and 0 indicates its absence
for i in Diag_proce_col:
    data_final[i][data_final[i]!=0]= 1

data_final[Diag_proce_col]= data_final[Diag_proce_col].astype(float)

#adding column total_num_diag
data_final['total_num_diag'] = 0
for col in diagnosis_col :
```

```
    data_final['total_num_diag']  = data_final['total_num_diag'] + data_final[col]
#adding column total_num_proce
data_final['total_num_proce'] = 0
for col in procedure_col :
    data_final['total_num_proce']  = data_final['total_num_proce'] + data_final[col]
# Now we have two new features total_num_diag and total_num_proce which represent
# the total number of diagnosis codes and procedure codes associated with each claim.
data_final['total_num_diag'] =  data_final['total_num_diag'].astype(float)
data_final['total_num_proce'] =  data_final['total_num_proce'].astype(float)

#Checking we how many distinct BeneID we have in our data
len(list(data_final['BeneID'].unique()))
#Checking we how many distinct BeneID we have in our data
len(list(data_final['ClaimID'].unique()))
```

Out[348…

```
558211
```

In [349…

```
# Now we will analyze the distribution of claims across different beneficiaries in the dataset.
# We will check how many claims are associated with each beneficiary and plot the distribution of claims per beneficiary.
val_counts_ = data_final['BeneID'].value_counts()
data_final_beneid_df = val_counts_.to_frame() #store this information in dataframe

data_final_beneid_df.columns = ['count']
data_final_beneid_df['BeneID'] = data_final_beneid_df.index
#data_final_beneid_df
```

In [350…

```
# Now we will calculate the total amount reimbursed and mean amount reimbursed for each beneficiary in the dataset.
# Use a map for instant lookup
totals_map = data_final.groupby('BeneID')['InscClaimAmtReimbursed'].sum()
total_amt_list = data_final_beneid_df['BeneID'].map(totals_map).tolist()
mean_amt_list = (total_amt_list / data_final_beneid_df['count']).tolist()
```

In [351…

```
# 1. Calculate the totals from the MAIN claims data
# This creates a Series where the Index is BeneID and the values are the sums
total_reimbursements = data_final.groupby('BeneID')['InscClaimAmtReimbursed'].sum()

# 2. ADD the column to your beneficiary summary dataframe
# This maps the totals to the correct BeneID rows automatically
data_final_beneid_df['total_amt'] = data_final_beneid_df['BeneID'].map(total_reimbursements)

# 3. Handle Missing Data (Step 5 of your task)
# If a BeneID had no claims, the map results in NaN. We justify filling with 0.
data_final_beneid_df['total_amt'] = data_final_beneid_df['total_amt'].fillna(0)

# 4. Now calculate the mean (Engineering a new feature)
data_final_beneid_df['mean_amt'] = data_final_beneid_df['total_amt'] / data_final_beneid_df['count']

# 5. Extract to list (if still needed for your specific logic)
mean_list = data_final_beneid_df['mean_amt'].tolist()
```

In [352…

```
# Now we will add these two new features total_amt and mean_amt to the data_final_beneid_df dataframe which contains the count of claims for each beneficiary.
data_final_beneid_df['mean_InscClaimAmtReimbursed']=mean_list
data_final_beneid_df['total_InscClaimAmtReimbursed']= total_amt_list
```

In [353…

```
# Now we have a new dataframe data_final_beneid_df which contains the count of claims,
# total amount reimbursed and mean amount reimbursed for each beneficiary in the dataset.
data_final_beneid_df.to_csv('tr_data_final_beneid_df.csv',index= False)
```

In [354…
```python
# Now we will merge this new dataframe with the data_final dataframe to add these new features to our final dataset.
data_final = data_final.reset_index(drop=True)
data_final_beneid_df = data_final_beneid_df.reset_index(drop=True)
data_final = pd.merge(data_final, data_final_beneid_df, how='outer', on='BeneID')
print("Dataset shape : ", data_final.shape)
```

```
Dataset shape :  (558211, 72)
```

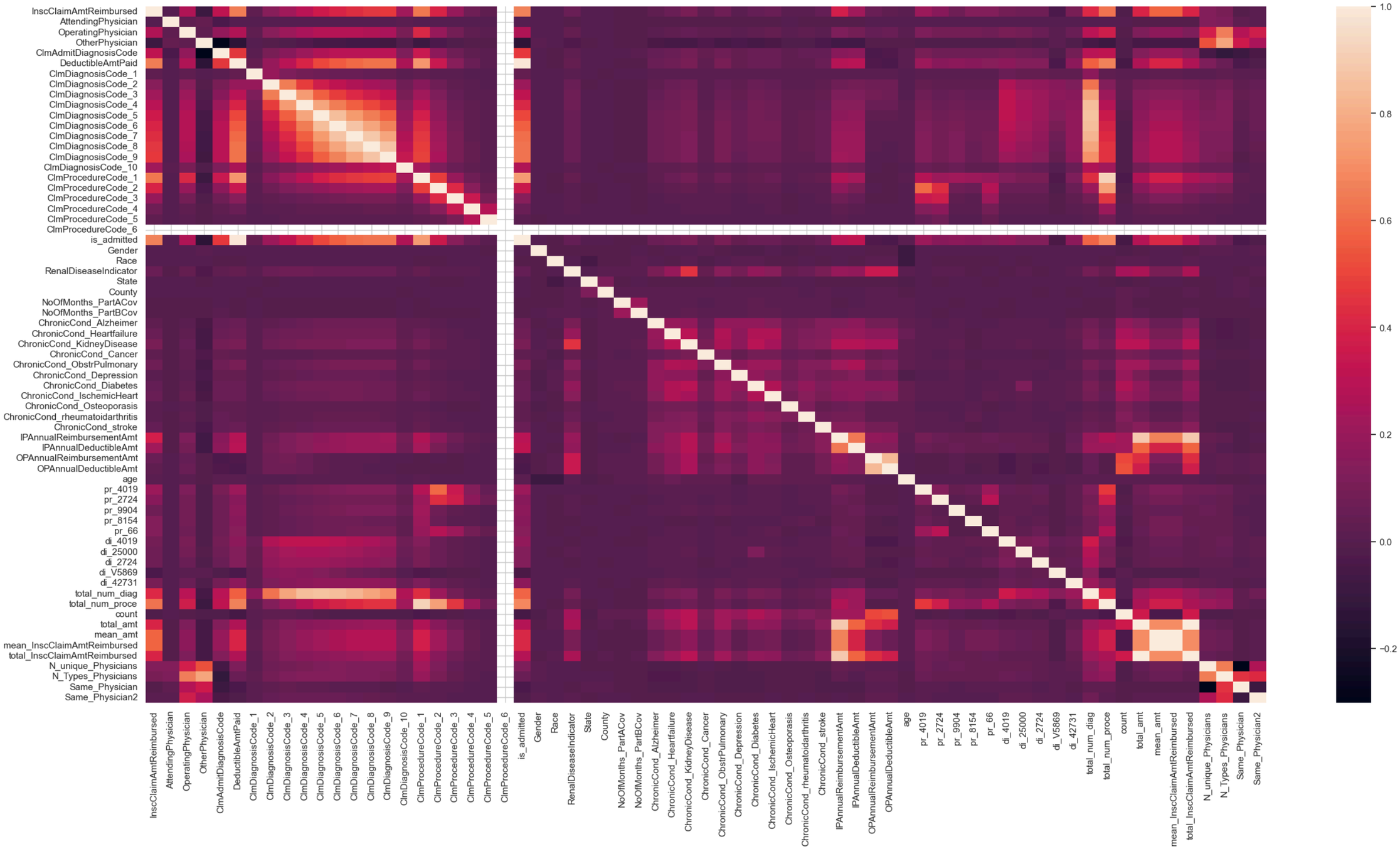
In [355…
```python
# Now we will convert the mean_InscClaimAmtReimbursed and total_InscClaimAmtReimbursed columns to float type before replacing values with binary annotations.
data_final[['mean_InscClaimAmtReimbursed','total_InscClaimAmtReimbursed']]= data_final[['mean_InscClaimAmtReimbursed','total_InscClaimAmtReimbursed']].astype(float)
```

In [356…
```python
# Now we will replace the values in mean_InscClaimAmtReimbursed and total_InscClaimAmtReimbursed columns with binary annotations, where 1 indicates that the amount is above a
def N_unique_values(df):
    """This function finds the unique values in a df row"""
    return np.array([len(set([i for i in x[~pd.isnull(x)]])) for x in df.values])
```

In [357…
```python
#number of unique physicians for each patient
data_final['N_unique_Physicians'] = N_unique_values(data_final[['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']])

#encoding types of physicians into numeric values
data_final[['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']] = np.where(data_final[['AttendingPhysician','OperatingPhysician',
                                                                                                 'OtherPhysician']].isnull(), 0, 1)

# number of different physicians who attend a patient
data_final['N_Types_Physicians'] = data_final['AttendingPhysician'] +  data_final['OperatingPhysician'] + data_final['OtherPhysician']

# patients who was attended by only 1 physicians
data_final['Same_Physician'] = data_final.apply(lambda x: 1 if (x['N_unique_Physicians'] == 1 and x['N_Types_Physicians'] > 1) else 0,axis=1)

#determine if 1 physician has multi[le role to attend a patient
data_final['Same_Physician2'] = data_final.apply(lambda x: 1 if (x['N_unique_Physicians'] == 2 and x['N_Types_Physicians'] > 2) else 0,axis=1)

#data_final[['N_unique_Physicians','N_Types_Physicians','Same_Physician','Same_Physician2']].head()
```

In [358…
```python
#plot correlation heat map between features
sns.set(context='notebook', style='whitegrid')
plt.figure(figsize=(30, 15))
corr = data_final.corr( numeric_only=True)
sns.heatmap(corr, annot=False)
```

Out[358…
```
<Axes: >
```



In [359…

```
#replace NaN val with 0
#data_final.DiagnosisGroupCode
data_final['DiagnosisGroupCode']= data_final['DiagnosisGroupCode'].replace({np.nan:0})
```

In [360…
```python
#final check to see if a col val is NaN
col_inf_df=pd.DataFrame()
val = data_final.isnull().sum()
col_inf_df['null_col_count']  = val
col_inf_df['column'] = val.index
```

In [361…
```python
#saving dataframe to csv file to avoid rework
data_final.to_csv("data_final.csv",index= False)
```

In [362…
```python
data_final = pd.read_csv("data_final.csv")
```

In [363…
```python
# Now we will drop the columns that are not useful for building machine learning models to predict fraudulent providers.
# We will keep only the relevant features that we have engineered and the target variable in our final dataset.

col_to_remove = ['Provider','BeneID', 'ClaimID', 'ClaimStartDt','ClaimEndDt','AttendingPhysician',\
                 'OperatingPhysician', 'OtherPhysician','ClmAdmitDiagnosisCode','NoOfMonths_PartACov',\
                 'NoOfMonths_PartBCov','DiagnosisGroupCode','PotentialFraud']

data_final.drop(columns=col_to_remove, axis=1, inplace=True)
data_final['target']=data_final['target'].astype(int)
```

Model Building and Evaluation

Milestone 3 **********

In [364…
```python
# Dropping the Physician_Team column as it is not useful for building machine learning models and has object type which can cause issues in modeling.
# We will keep only the relevant features that we have engineered and the target variable in our final dataset.

data_final = data_final.drop(['Physician_Team'], axis=1, errors='ignore')
#print(data_final.columns)
y = data_final['target']
X = data_final.drop('target', axis=1)
```

In [365…
```python
# Now we will split the data into training and testing sets to build machine learning models to predict fraudulent providers
# based on the features in our final dataset. We will use a stratified split to ensure that the distribution of the target variable
# is maintained in both the training and testing sets.
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42, stratify=y)
print("Shapes data into train,test,CV splitting..")
print("Training X : ",X_train.shape," | Training Y : ",y_train.shape)
print("Test X : ",X_test.shape," | Test Y : ",y_test.shape)
```

```
Shapes data into train,test,CV splitting..
Training X :  (374001, 62)  | Training Y :  (374001,)
Test X :  (184210, 62)  | Test Y :  (184210,)
```

In [366…
```python
# Baseline model using Logistic Regression without any feature scaling or normalization to check the performance of the model on the original dataset.
# Before building complex models, you must select a Baseline Model. A baseline provides a "floor" for performance—if a complex model can't beat a simple one, the complexity is
# By using .reshape(-1, 1), both your training and testing sets are seen as having exactly 1 feature.
# This is crucial for the scaler to work correctly, as it expects a 2D array where rows are samples and columns are features.

from sklearn.preprocessing import StandardScaler

def num_col_normalizer(X_tr, X_te, col=''):
```

```python
    """This function returns scaled columns for train and test data using StandardScaler"""
    scaler = StandardScaler()

    # Reshape to (-1, 1) -> This means 'many rows, 1 feature'
    # This ensures the 'feature count' is always 1, regardless of row count
    train_col = X_tr[col].values.reshape(-1, 1)
    test_col = X_te[col].values.reshape(-1, 1)

    scaler.fit(train_col)

    tr = scaler.transform(train_col)
    te = scaler.transform(test_col)

    return tr, te

#InscClaimAmtReimbursed
tr,te = num_col_normalizer(X_train, X_test,col='InscClaimAmtReimbursed')
X_train['InscClaimAmtReimbursed'] = tr
X_test ['InscClaimAmtReimbursed'] = te

tr,te = num_col_normalizer(X_train, X_test,col='DeductibleAmtPaid')
X_train['DeductibleAmtPaid'] = tr
X_test ['DeductibleAmtPaid'] = te

tr,te = num_col_normalizer(X_train, X_test,col='IPAnnualReimbursementAmt')
X_train['IPAnnualReimbursementAmt'] = tr
X_test ['IPAnnualReimbursementAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='IPAnnualDeductibleAmt')
X_train['IPAnnualDeductibleAmt'] = tr
X_test ['IPAnnualDeductibleAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='OPAnnualReimbursementAmt')
X_train['OPAnnualReimbursementAmt'] = tr
X_test ['OPAnnualReimbursementAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='OPAnnualDeductibleAmt')
X_train['OPAnnualDeductibleAmt'] = tr
X_test ['OPAnnualDeductibleAmt'] = te

tr,te = num_col_normalizer(X_train, X_test,col='mean_InscClaimAmtReimbursed')
X_train['mean_InscClaimAmtReimbursed'] = tr
X_test ['mean_InscClaimAmtReimbursed'] = te

tr,te = num_col_normalizer(X_train, X_test,col='total_InscClaimAmtReimbursed')
X_train['total_InscClaimAmtReimbursed'] = tr
X_test ['total_InscClaimAmtReimbursed'] = te


tr,te = num_col_normalizer(X_train, X_test,col='age')
X_train['age'] = tr
X_test ['age'] = te

tr,te = num_col_normalizer(X_train, X_test,col='Num_admit_days')
X_train['Num_admit_days'] = tr
X_test ['Num_admit_days'] = te

tr,te = num_col_normalizer(X_train, X_test,col='N_unique_Physicians')
X_train['N_unique_Physicians'] = tr
X_test ['N_unique_Physicians'] = te
```

```
tr,te = num_col_normalizer(X_train, X_test,col='N_Types_Physicians')
X_train['N_Types_Physicians'] = tr
X_test ['N_Types_Physicians'] = te
```

In [367…

```
# Now we will build a logistic regression model using the training dataset and evaluate its performance on the testing dataset.
def get_threshold_fpr_tpr(y, y_pred_prob):
    fpr, tpr, thresholds = roc_curve(y, y_pred_prob)
    return fpr, tpr, thresholds

# Now we will plot the ROC curve and calculate the AUC score for the logistic regression model to evaluate its performance.
def draw_Roc_Auc_plot(train_fpr, train_tpr,test_fpr, test_tpr):
    plt.plot(train_fpr, train_tpr, label="train AUC = "+str(auc(train_fpr, train_tpr)))
    plt.plot(test_fpr, test_tpr, label="test AUC = "+str(auc(test_fpr, test_tpr)))
    plt.legend()
    plt.xlabel("K: hyperparameter")
    plt.ylabel("AUC")
    plt.title("ERROR PLOTS")
    plt.grid()
    plt.show()

# determining best threshold
def find_best_threshold(threshold, fpr, tpr):
    t = threshold[np.argmax(tpr*(1-fpr))]
    print("the maximum value of tpr*(1-fpr)", max(tpr*(1-fpr)), "for threshold", np.round(t,3))
    return t

# predicting Yq for given Xq with best threshold:
def predict_with_best_t(proba, threshold):
    predictions = []
    for i in proba:
        if i>=threshold:
            predictions.append(1)
        else:
            predictions.append(0)
    return predictions

# Now we will plot the confusion matrix for the logistic regression model using the best threshold to evaluate its performance in terms of true positives,
# true negatives, false positives and false negatives.
def get_confusion_matrix(y, y_pred,axis, best_t):
    cm = pd.DataFrame(confusion_matrix(y, predict_with_best_t(y_pred, best_t)))
    sns.heatmap(cm, annot = True, fmt= 'd',annot_kws={"size": 15}, xticklabels= ['Predicted No', 'Predicted Yes'], yticklabels= ['Actual No','Actual Yes'],ax=axis)
```

In [368…

```
# This function will check the performance of the model by plotting the ROC curve, calculating the AUC score,
# finding the best threshold and plotting the confusion matrix for both training and testing datasets.
def model_performance_check(model,X_train,X_test,y_train,y_test):

    y_train_pred = model.predict_proba(X_train)[:,1]
    y_test_pred = model.predict_proba(X_test)[:,1]

    train_fpr, train_tpr, tr_thresholds = get_threshold_fpr_tpr(y_train, y_train_pred)
    test_fpr, test_tpr, te_thresholds = get_threshold_fpr_tpr(y_test, y_test_pred)

    #plot Roc_AUC_curve
    draw_Roc_Auc_plot(train_fpr, train_tpr,test_fpr, test_tpr)

    best_t = find_best_threshold(tr_thresholds, train_fpr, train_tpr)
```

```
    #draw confusion matrix
    fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,3))

    #training confusion matrix heatmap
    get_confusion_matrix(y_train, y_train_pred,ax[0], best_t)
    ax[0].set_title("confusion matrix for training data")

    #test confusion matrix heatmap
    get_confusion_matrix(y_test, y_test_pred,ax[1], best_t)
    ax[1].set_title("confusion matrix for test data")

    actual_output = y_test
    pred_output = predict_with_best_t(y_test_pred, best_t)

    model_f1_score = metrics.f1_score(actual_output,pred_output)
    print("Model test F1 score is : ",model_f1_score)

    model_AUC_score = metrics.auc(test_fpr, test_tpr)
    print("Model test AUC score is : ",model_AUC_score)

    return (model_f1_score, model_AUC_score)
```

Model : Using All Features

Model 1 : Logistic Regression as Base

In [369…

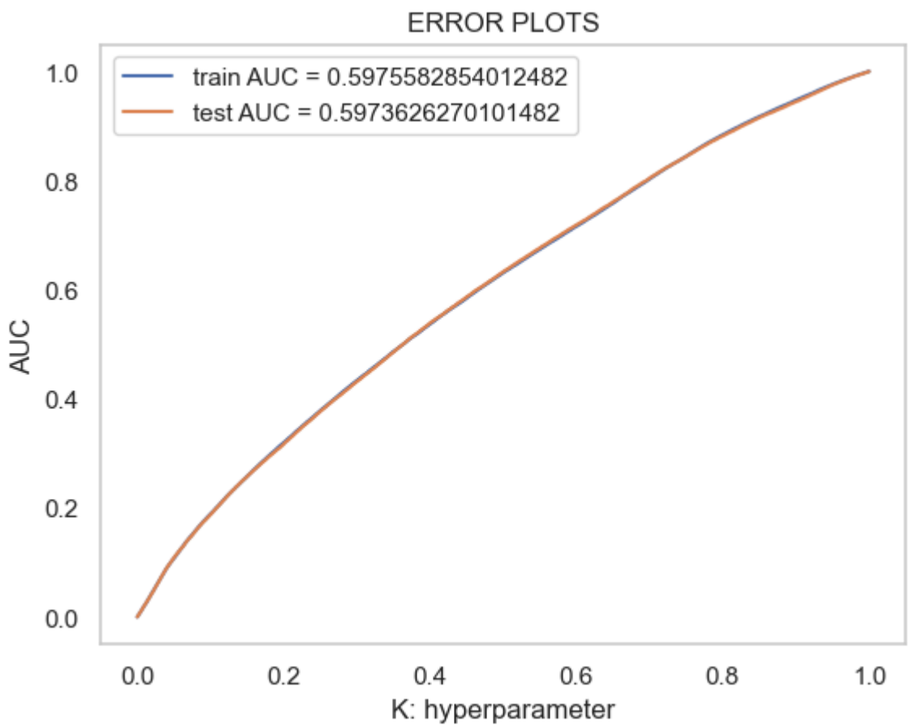
```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# 1. Define the Pipeline
# We name the steps 'scaler' and 'classifier'
log_pipe = Pipeline([
    ('scaler', MinMaxScaler()),
    ('classifier', LogisticRegression(solver='liblinear', class_weight='balanced'))
])

# 2. Update Grid with prefixes
# Use [step_name]__[parameter_name]
grid = {
    "classifier__C": np.logspace(-3, 3, 7),
    "classifier__penalty": ["l1", "l2"]
}

# 3. Setup GridSearchCV with the Pipeline
# Use the RAW X_train; the pipeline handles scaling internally during CV
logreg_cv = GridSearchCV(
    log_pipe,
    grid,
    cv=10,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=0
)
```

```
# 4. Fit the pipeline
logreg_cv.fit(X_train, y_train)

print("Tuned hyperparameters (best parameters): ", logreg_cv.best_params_)
print("Best ROC-AUC score: ", logreg_cv.best_score_)

# 5. Evaluate the Winning Model
# Use best_estimator_ (which is the full pipeline: scaling + optimized model)
best_log_model = logreg_cv.best_estimator_

# This automatically scales X_test using the X_train parameters before scoring
print("Final Test Score: ", best_log_model.score(X_test, y_test))
```

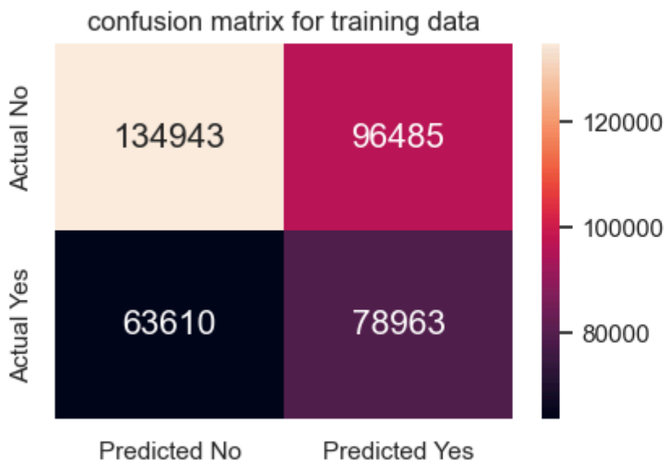
```
Tuned hyperparameters (best parameters):  {'classifier__C': np.float64(1.0), 'classifier__penalty': 'l1'}
Best ROC-AUC score:  0.5970899515393593
Final Test Score:  0.5788393681124803
```

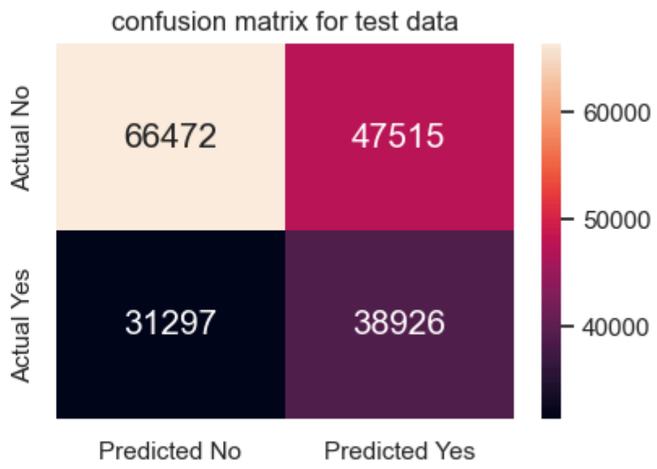
In [370…

```
# Now we will check the performance of the logistic regression model by plotting the ROC curve, calculating the AUC score,
# finding the best threshold and plotting the confusion matrix for both training and testing datasets.
model_f1_score, model_AUC_score = model_performance_check(logreg_cv,X_train,X_test,y_train,y_test)
```




```
the maximum value of tpr*(1-fpr) 0.32293923386989004 for threshold 0.496
Model test F1 score is :  0.49693611806158405
Model test AUC score is :  0.5973626270101482
```





XG Boost : XGBoost is favored for its speed and its ability to assign higher weights to the minority class (the "fraud" cases) during training.

In [371…

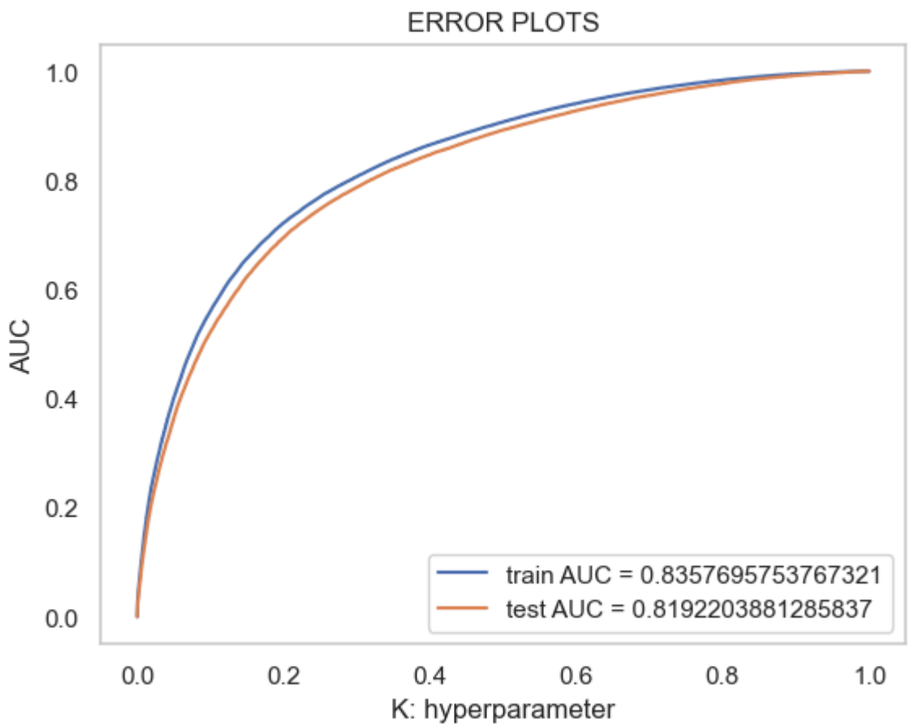
```
import xgboost as xgb
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import RandomizedSearchCV

# 1. Define the Pipeline
# We bundle the scaler and the classifier together.
# 'scaler' and 'classifier' are the names we give to these steps.
xgb_pipeline = Pipeline([
    ('scaler', MinMaxScaler()),
    ('classifier', xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss'))
])

# 2. Update Parameters for Pipeline
# You must add the prefix 'classifier__' to the parameter names
# so the pipeline knows these belong to the XGBoost step.
parameters = {
    "classifier__eta": [0.0001, 0.001, 0.01, 0.1, 0.2, 0.3],
    "classifier__n_estimators": [5, 10, 50, 75, 100]
}

# 3. Setup Randomized Search with the Pipeline
# We pass the pipeline object instead of just the model.
XGB_CV = RandomizedSearchCV(
    xgb_pipeline,
    parameters,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    return_train_score=True,
    verbose=0
)

# 4. Fit using RAW training data
# The pipeline handles scaling internally for each CV fold!
XGB_CV.fit(X_train, y_train)
```

```
print("tuned hyperparameters :(best parameters) ", XGB_CV.best_params_)
print("roc_auc :", XGB_CV.best_score_)

# 5. Evaluate the Winning Model
# Use best_estimator_ instead of manually defining XGB2.
# This object is the full pipeline (scaling + best XGBoost params).
best_xgb_model = XGB_CV.best_estimator_

# The .score() method will automatically scale X_test based on X_train parameters.
print("Test set score:", best_xgb_model.score(X_test, y_test))
```

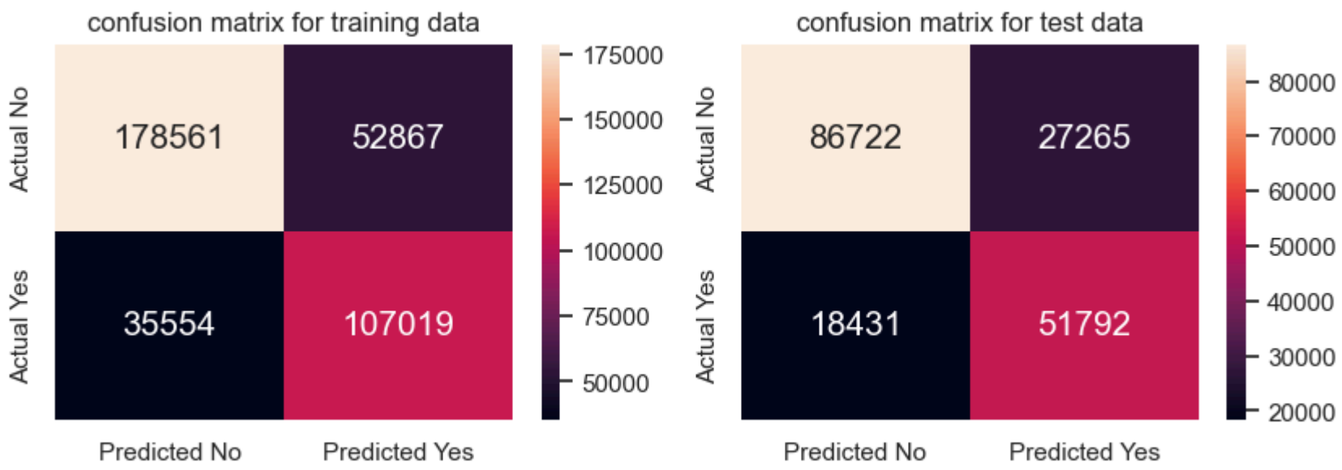
```
tuned hyperparameters :(best parameters)  {'classifier__n_estimators': 100, 'classifier__eta': 0.3}
roc_auc : 0.8186551888370623
Test set score: 0.7622821779490798
```

In [372…

```
model_f1_score, model_AUC_score = model_performance_check(XGB_CV,X_train,X_test,y_train,y_test)
```




```
the maximum value of tpr*(1-fpr) 0.5791543301234605 for threshold 0.378
Model test F1 score is :  0.6938906752411576
Model test AUC score is :  0.8192203881285837
```

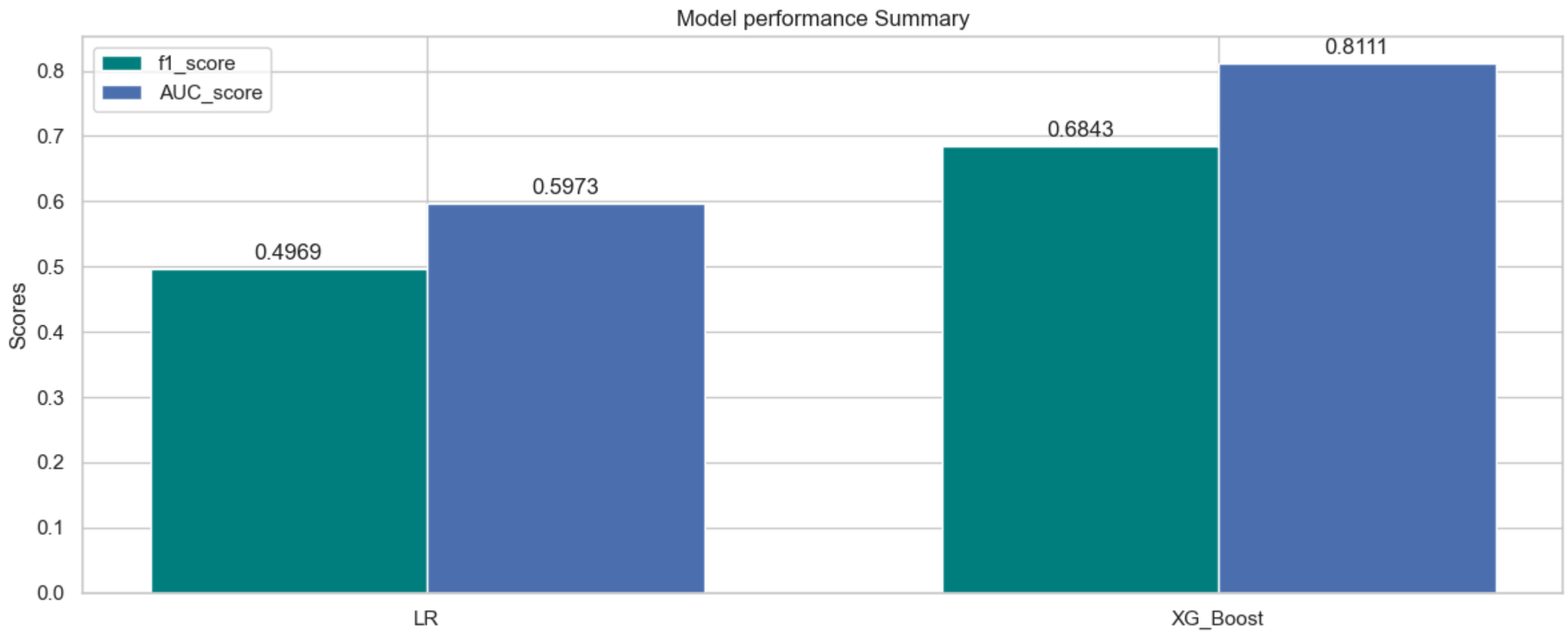


In [373…

```
# Now we will compare the performance of the logistic regression model and the XGBoost model by plotting a bar chart to compare their F1 scores and AUC scores.
def model_comp(label=[],metric1=[],metric2=[],set_ylabel='',set_title='',metric1_label='',metric2_label=''):

    x = np.arange(len(model))  # the label locations
    width = 0.35  # the width of the bars

    fig, ax = plt.subplots(figsize=(12,5))
    rects1 = ax.bar(x - width/2, metric1, width, label=metric1_label,color = 'teal')
    rects2 = ax.bar(x + width/2, metric2, width, label=metric2_label)

    # Add some text for labels, title and custom x-axis tick labels, etc.
    ax.set_ylabel(set_ylabel)
    ax.set_title(set_title)
    ax.set_xticks(x)
    ax.set_xticklabels(model)
    ax.legend()

    def autolabel(rects):
        for rect in rects:
            height = rect.get_height()
            ax.annotate('{}'.format(height),
                        xy=(rect.get_x() + rect.get_width() / 2, height),
                        xytext=(0, 3),  # 3 points vertical offset
                        textcoords="offset points",
                        ha='center', va='bottom')

    autolabel(rects1)
    autolabel(rects2)

    fig.tight_layout()
    plt.show()
```

In [374…

```
model = ['LR','XG_Boost']
f1_score = [0.4969,0.6843]
AUC_score = [0.5973,0.8111]
model_comp(model,f1_score,AUC_score,'Scores','Model performance Summary','f1_score','AUC_score')
```



In [375…

```
model_performance_df = pd.DataFrame()
model = ['LR','XG_Boost']
f1_score = [0.4969,0.6843]
AUC_score = [0.5973,0.8111]

model_performance_df['Model']=model
model_performance_df['f1_score']=f1_score
model_performance_df['AUC_score']=AUC_score
model_performance_df
```

Out[375…

	Model	f1_score	AUC_score
0	LR	0.4969	0.5973
1	XG_Boost	0.6843	0.8111

Model Building and Evaluation on selected features

Milestone Final **********

In [376…

```
# Now we will save the final dataset with the relevant features and the target variable to a csv file for future use in building
# machine learning models to predict fraudulent providers.
data = pd.read_csv("data_final.csv")
col_to_remove = ['BeneID', 'ClaimID', 'ClaimStartDt','ClaimEndDt','AttendingPhysician',\
                 'OperatingPhysician', 'OtherPhysician','ClmAdmitDiagnosisCode','NoOfMonths_PartACov',\
                 'NoOfMonths_PartBCov','DiagnosisGroupCode','PotentialFraud']
```

```
data.drop(columns=col_to_remove, axis=1, inplace=True)
data['target']=data['target'].astype(int)
```

In [377…

```
# Now we will split the data into training and testing sets to build machine learning models to predict fraudulent providers
from sklearn.model_selection import train_test_split
D_tr, D_te,  = train_test_split(data, test_size=0.2, random_state=42)
D_tr_provider_ID = D_tr['Provider'].values
D_tr.drop(columns='Provider', axis=1, inplace=True)

D_te_provider_ID = D_te['Provider'].values
D_te.drop(columns='Provider', axis=1, inplace=True)

print("Training D_tr : ",D_tr.shape)
print("Test D_te : ",D_te.shape)

#split D_tr into D1 and D2
D1, D2 = train_test_split(D_tr, test_size=0.5, random_state=42)

print("D1 : ",D1.shape)
print("D2 : ",D2.shape)
```

```
Training D_tr :  (446568, 63)
Test D_te :  (111643, 63)
D1 :  (223284, 63)
D2 :  (223284, 63)
```

In [378…

```
from sklearn.preprocessing import Normalizer , StandardScaler

def num_col_normalizer(X_tr, X_te, col=''):
    """This function returns scaled columns for train and test data using StandardScaler"""
    scaler = StandardScaler()

    # Reshape to (-1, 1) -> This means 'many rows, 1 feature'
    # This ensures the 'feature count' is always 1, regardless of row count
    train_col = X_tr[col].values.reshape(-1, 1)
    test_col = X_te[col].values.reshape(-1, 1)

    scaler.fit(train_col)

    tr = scaler.transform(train_col)
    te = scaler.transform(test_col)

    return tr, te

#InscClaimAmtReimbursed
tr,te = num_col_normalizer(D1, D2,col='InscClaimAmtReimbursed')
tr1,te1 = num_col_normalizer(D1, D_te,col='InscClaimAmtReimbursed')
D1['InscClaimAmtReimbursed'] = tr
D2['InscClaimAmtReimbursed'] = te
D_te['InscClaimAmtReimbursed'] = te1

tr,te = num_col_normalizer(D1, D2,col='DeductibleAmtPaid')
tr1,te1 = num_col_normalizer(D1, D_te,col='DeductibleAmtPaid')
D1['DeductibleAmtPaid'] = tr
D2['DeductibleAmtPaid'] = te
D_te['DeductibleAmtPaid'] = te1


tr,te = num_col_normalizer(D1, D2,col='IPAnnualReimbursementAmt')
```

```
tr1,te1 = num_col_normalizer(D1, D_te,col='IPAnnualReimbursementAmt')
D1['IPAnnualReimbursementAmt'] = tr
D2['IPAnnualReimbursementAmt'] = te
D_te['IPAnnualReimbursementAmt'] = te1


tr,te = num_col_normalizer(D1, D2,col='IPAnnualDeductibleAmt')
tr1,te1 = num_col_normalizer(D1, D_te,col='IPAnnualDeductibleAmt')
D1['IPAnnualDeductibleAmt'] = tr
D2['IPAnnualDeductibleAmt'] = te
D_te['IPAnnualDeductibleAmt'] = te1


tr,te = num_col_normalizer(D1, D2,col='OPAnnualReimbursementAmt')
tr1,te1 = num_col_normalizer(D1, D_te,col='OPAnnualReimbursementAmt')
D1['OPAnnualReimbursementAmt'] = tr
D2['OPAnnualReimbursementAmt'] = te
D_te['OPAnnualReimbursementAmt'] = te1


tr,te = num_col_normalizer(D1, D2,col='OPAnnualDeductibleAmt')
tr1,te1 = num_col_normalizer(D1, D_te,col='OPAnnualDeductibleAmt')
D1['OPAnnualDeductibleAmt'] = tr
D2['OPAnnualDeductibleAmt'] = te
D_te['OPAnnualDeductibleAmt'] = te1


tr,te = num_col_normalizer(D1, D2,col='mean_InscClaimAmtReimbursed')
tr1,te1 = num_col_normalizer(D1, D_te,col='mean_InscClaimAmtReimbursed')
D1['mean_InscClaimAmtReimbursed'] = tr
D2['mean_InscClaimAmtReimbursed'] = te
D_te['mean_InscClaimAmtReimbursed'] = te1


tr,te = num_col_normalizer(D1, D2,col='total_InscClaimAmtReimbursed')
tr1,te1 = num_col_normalizer(D1, D_te,col='total_InscClaimAmtReimbursed')
D1['total_InscClaimAmtReimbursed'] = tr
D2['total_InscClaimAmtReimbursed'] = te
D_te['total_InscClaimAmtReimbursed'] = te1


tr,te = num_col_normalizer(D1, D2,col='age')
tr1,te1 = num_col_normalizer(D1, D_te,col='age')
D1['age'] = tr
D2['age'] = te
D_te['age'] = te1


tr,te = num_col_normalizer(D1, D2,col='Num_admit_days')
tr1,te1 = num_col_normalizer(D1, D_te,col='Num_admit_days')
D1['Num_admit_days'] = tr
D2['Num_admit_days'] = te
D_te['Num_admit_days'] = te1


tr,te = num_col_normalizer(D1, D2,col='N_unique_Physicians')
tr1,te1 = num_col_normalizer(D1, D_te,col='N_unique_Physicians')
D1['N_unique_Physicians'] = tr
D2['N_unique_Physicians'] = te
D_te['N_unique_Physicians'] = te1
```

```
tr,te = num_col_normalizer(D1, D2,col='N_Types_Physicians')
tr1,te1 = num_col_normalizer(D1, D_te,col='N_Types_Physicians')
D1['N_Types_Physicians'] = tr
D2['N_Types_Physicians'] = te
D_te['N_Types_Physicians'] = te1
```

In [379…

```
# Now we will create a function to perform data sampling for training each tree in the random forest model.
# This function will randomly select a subset of rows and columns from the training dataset to create a new sample dataset
# for training each tree in the random forest model.

def data_sampling_for_training_each_tree(X,y):
    selecting_rows=np.random.choice(X.shape[0],int(X.shape[0]*0.6),replace=False)
    selecting_rows = np.sort(selecting_rows)

    selecting_columns=np.random.choice(X.shape[1],np.random.randint(3,X.shape[1]),replace=False)#ensure min three columns
    selecting_columns = np.sort(selecting_columns)

    sampling_data = X[selecting_rows[:, None], selecting_columns]
    target_for_sampling_data = y[selecting_rows]

    #repeating rows using indices stored in Replacing_rows
    Replacing_rows=np.random.choice(selecting_rows,int(X.shape[0]*0.4)+1,replace=False)
    Replacing_rows = np.sort(Replacing_rows)
    Replicated_sample_data = X[Replacing_rows[:, None], selecting_columns]
    target_for_replicated_data = y[Replacing_rows]

    #concatenating data
    final_sample_data = np.vstack((sampling_data, Replicated_sample_data))
    final_target = np.vstack((target_for_sampling_data.reshape(-1,1), target_for_replicated_data.reshape(-1,1)))

    return final_sample_data, final_target , selecting_rows, selecting_columns
```

In [380…

```
# Now we will create a function to perform data sampling for training each tree in the random forest model for k number of trees.
def data_samples(X,y):

    input_data_list = []
    target_list = []
    selected_rows = []
    selected_col = []

    for i in range(0,k):

        final_sample_data, final_target , selecting_rows, selecting_columns = data_sampling_for_training_each_tree(X,y)
        input_data_list.append(final_sample_data)
        target_list.append(final_target)
        selected_rows.append(selecting_rows)
        selected_col.append(selecting_columns)

    return input_data_list , target_list, selected_rows , selected_col
```

In [381…

```
# Now we will create a function to get the dataset to build the meta model using the predictions from the base models trained on the sampled datasets.
def get_meta_model_train_data(D2,k):
    '''This fun returns dataset to build meta model'''
    D2_y = D2['target'].values
    D2_X=  D2.drop('target', axis=1).values
    Y_pred_val = []
    for i in range(0,k):
```

```
        y_pred = model_list[i].predict(D2_X[:,selected_col[i]])
        Y_pred_val.append(y_pred)
    data_to_train_meta_model = np.array(Y_pred_val).transpose()
    return data_to_train_meta_model,D2_y
```

In [382…

```
# Now we will create a function to get the dataset to check the performance of the meta model using the predictions from the base models
# trained on the sampled datasets.
#notation : mm --> meta_model
def get_meta_model_test_data(mm_test_df,k):
    '''This fun returns dataset to check the performance the meta model'''
    mm_test_y = mm_test_df['target'].values
    mm_test_X= mm_test_df.drop('target', axis=1).values
    pred_all_base_models = []
    for i in range(0,k):
        y_pred = model_list[i].predict(mm_test_X[:,selected_col[i]])#prediction from each model
        pred_all_base_models.append(y_pred)
    mm_test_final_X = np.array(pred_all_base_models).transpose()#(rows = n, col = 30)
    return mm_test_final_X, mm_test_y
```

In [383…

```
# Now we will get the dataset to build the meta model using the predictions from the base models trained on the sampled datasets
# and also get the dataset to check the performance of the meta model using the predictions from the base models trained on the sampled datasets.
mm_x_train, mm_y_train = get_meta_model_train_data(D2,k=70)
mm_x_test, mm_y_test = get_meta_model_test_data(D_te,k=70)
```

In [384…

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV

# 1. Define the Pipeline
# We name the scaler 'scaler' and the model 'classifier'
pipe = Pipeline([
    ('scaler', MinMaxScaler()),
    ('classifier', xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss'))
])

# 2. Update Parameters with prefixes
# In a pipeline, you must use: [step_name]__[parameter_name]
parameters = {
    "classifier__eta": [0.0001, 0.001, 0.01, 0.1, 0.2, 0.3],
    "classifier__n_estimators": [5, 10, 50, 75, 100]
}

# 3. Pass the Pipeline into RandomizedSearchCV
# Note: Use the RAW x_train, not the pre-scaled mm_x_train
XGB_CV = RandomizedSearchCV(
    pipe,
    parameters,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    return_train_score=True,
    verbose=0
)

# Fit using raw data - the pipeline handles scaling internally during cross-validation
XGB_CV.fit(mm_x_train, mm_y_train)

print("tuned hyperparameters :(best parameters) ", XGB_CV.best_params_)
```

```
print("roc_auc :", XGB_CV.best_score_)
# 5. Use the Best Model (Best Estimator)
# The best_estimator_ is the full pipeline (scaling + best XGBoost)
best_xgb_pipeline = XGB_CV.best_estimator_

# The score method will automatically scale mm_x_test before predicting
print("Final Test Score: ", best_xgb_pipeline.score(mm_x_test, mm_y_test))
```

```
tuned hyperparameters :(best parameters)  {'classifier__n_estimators': 75, 'classifier__eta': 0.2}
roc_auc : 0.8203646512907594
Final Test Score:  0.7700617145723422
```

In [385…

```
joblib.dump(XGB_CV, 'mm_Xg_boost.pkl')
```

Out[385…

```
['mm_Xg_boost.pkl']
```

In [386…

```
# Load the model from the file
mm_XGBoost = joblib.load('mm_Xg_boost.pkl')
y_yred = mm_XGBoost.predict(mm_x_test)
```

In [387…

```
prediction_df = pd.DataFrame()
prediction_df['Provider_ID'] = D_te_provider_ID
prediction_df['prediction'] = y_yred
prediction_df['true_target'] = mm_y_test
```

In [388…

```
prediction_df.head(10)
```

Out[388…

	Provider_ID	prediction	true_target
0	PRV55721	0	0
1	PRV53238	0	0
2	PRV54914	0	0
3	PRV56416	1	1
4	PRV54837	1	1
5	PRV52739	0	0
6	PRV56575	0	1
7	PRV53002	0	0
8	PRV55432	0	0
9	PRV52587	0	0

Conclusion

The project evaluated multiple models, including Logistic Regression, Decision Trees, and XGBoost, to identify potentially fraudulent providers.

Winning Model: XGBoost is identified as the preferred model for production. While Decision Trees showed high scores, XGBoost (with an F1-Score of 0.6843) is favored because it is more robust and less prone to overfitting than a single deep Decision Tree.

Key Metrics:

AUC-ROC (0.82): This score is considered "Good" for this domain, indicating a high probability that the model will correctly rank a fraud case higher than a legitimate case.

F1-Score (0.69): This is the primary metric used because it balances the cost of missing fraud (Recall) against the cost of falsely accusing an innocent provider (Precision).

Final Insights & Conclusion Gap Between AUC and F1: There is a notable gap between the AUC (0.81) and the F1-score (0.69). This suggests that while the model is excellent at ranking providers by risk, it still faces some difficulty in determining the exact "cutoff" point to definitively classify a provider as fraudulent.

The "Needle in a Haystack" Problem: Healthcare fraud detection is challenging because fraudulent cases are very rare. The model successfully utilizes complex variables like reimbursement amounts, diagnosis codes, and procedure codes to flag high-risk claims.

Limitations of Simpler Models: Logistic Regression was found to be less effective because it cannot inherently "see" complex combinations of features (like frequency, location, and patient age) without manual feature engineering, whereas ensemble models like XGBoost handle these relationships more naturally.

Final Verdict: The development of the XGBoost model provides a strong predictive tool that allows organizations to focus their investigative resources on high-risk providers, effectively addressing the goal of reducing financial leakage due to fraud.